Library Management System

SWEN-383 Software Design Principles and Patterns

Project Part 1: Definition & Design Baseline

Team: The Principled Four

RIT Kosovo, Fall 2026 · 26 October 2026

Erion Elezaj

Elion Elezaj

Lorand Pervizaj

Dren Veliu

1. Project Description

A Library Management System that gives functionality for students, librarians and assistant librarians, offering services and managing those services.

Students can search the digital library catalog, check the availability of a book, and place a reservation for a book title. A reservation is for a book title, not a specific copy. When a copy is free, it is put on hold for the student who reserved it, and the student gets an email. The hold expires after the hold period if the student does not pick the book up; the copy is then put on hold for the next student in line, or goes back on the shelf if no one is waiting. If all copies are out, students wait in line in the order they reserved. A student can cancel a reservation at any time before picking up the book. Students can also walk into the library and borrow any copy that is on the shelf and not on hold.

The physical book must be handed out by a librarian or assistant librarian, who scans the copy. At that moment the system records the loan and gives a due date. Students return the book at the desk, where staff scan the copy again, and the system records the return. When a copy is returned and students are waiting for that title, the system puts it on hold for the next student in line and emails them. Students can see the status of their loans, reservations and fines: a reservation is waiting in line, on hold with an expiry date, completed, expired, or cancelled; a loan is on loan with a due date, overdue, or returned.

Once a day, the system checks for overdue loans and emails the students who have them, and it releases holds that have expired. If a book is returned late, the system calculates a fine based on how many days it is overdue. A librarian can waive the fine. Fines are paid in person at the desk, where a librarian records the payment.

Librarians hand out physical copies and record loans: they scan the copy at the desk, and the system records the loan and assigns a due date. They manage the physical copies of each book by adding copies and marking copies as damaged, and they can see the status of every individual copy (available, on hold, on loan, or damaged). They can add new books (author, title, ISBN), edit book information, and remove books. A book cannot be removed while any of its copies is on loan or on hold, or while students are waiting for it; a removed book is marked as withdrawn and its record is kept. Damaged copies are taken off the shelf but their record is kept, so loan history stays intact. A copy that is on loan cannot be marked as damaged until it is returned.

Assistant librarians work at the desk: they scan copies when handing them out (the system records the loan) and when they are returned (the system records the return and calculates any fine). Librarians and assistant librarians can view reservations and holds: the queue order, each student's position, and which copy is on hold for which student.

Librarians also configure the library settings: the loan period, the hold period, and the fine rate.

The catalog is digital, but the books are physical. A reservation is only a request. The librarian or assistant librarian physically hands the book over, and that's the moment the loan starts.

2. Scope

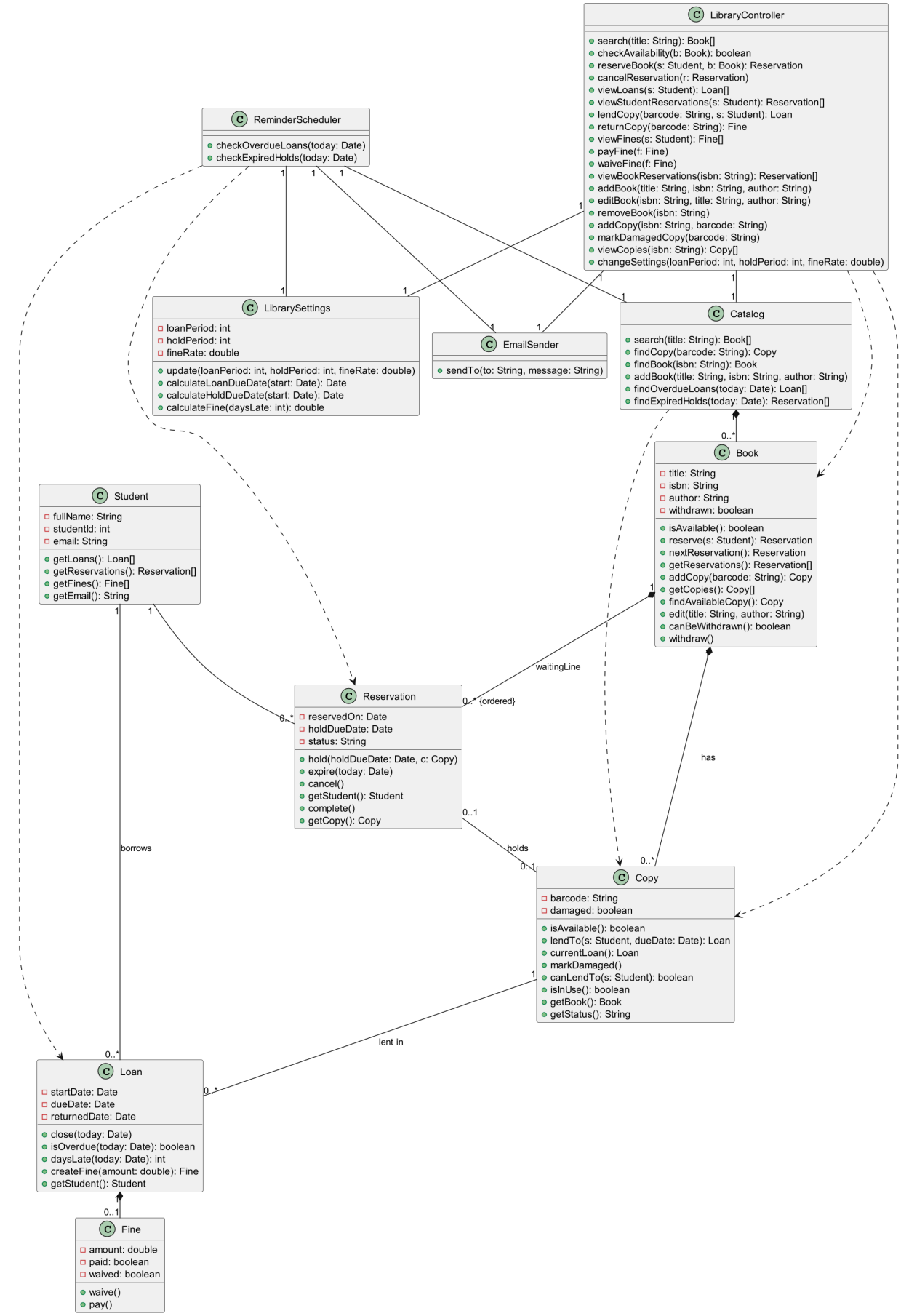
In scope

- Catalog search and availability check
- Reservations, handouts, loans, due dates, returns
- Students can cancel a reservation
- A hold is put on a reserved copy and expires after the hold period; the copy then goes on hold for the next student in line, or back on the shelf if no one is waiting
- Students can pick up copies without reserving, if it is on the shelf and not on hold
- Students wait in line in the order they reserved for books that have no available copies in the moment
- Loan, reservation and fine status view for students
- Fines for overdue books, calculated by the system; librarians can waive them
- Fines are paid in person at the desk, and a librarian records the payment in the system
- Book and physical copy management (add / edit / remove)
- A book cannot be removed while any of its copies is on loan or on hold, or while students are waiting for it; a removed book is marked as withdrawn and its record is kept
- Damaged copies are taken off the shelf and marked as damaged; their record is kept so the loan history stays intact
- Scan-based identification of copies
- Reservation overview for staff
- Email to students when a copy they reserved is available, and when a loan is overdue
- Librarians handle the configuration of library settings & policies such as loan period, hold period, and fine rate.

Out of scope

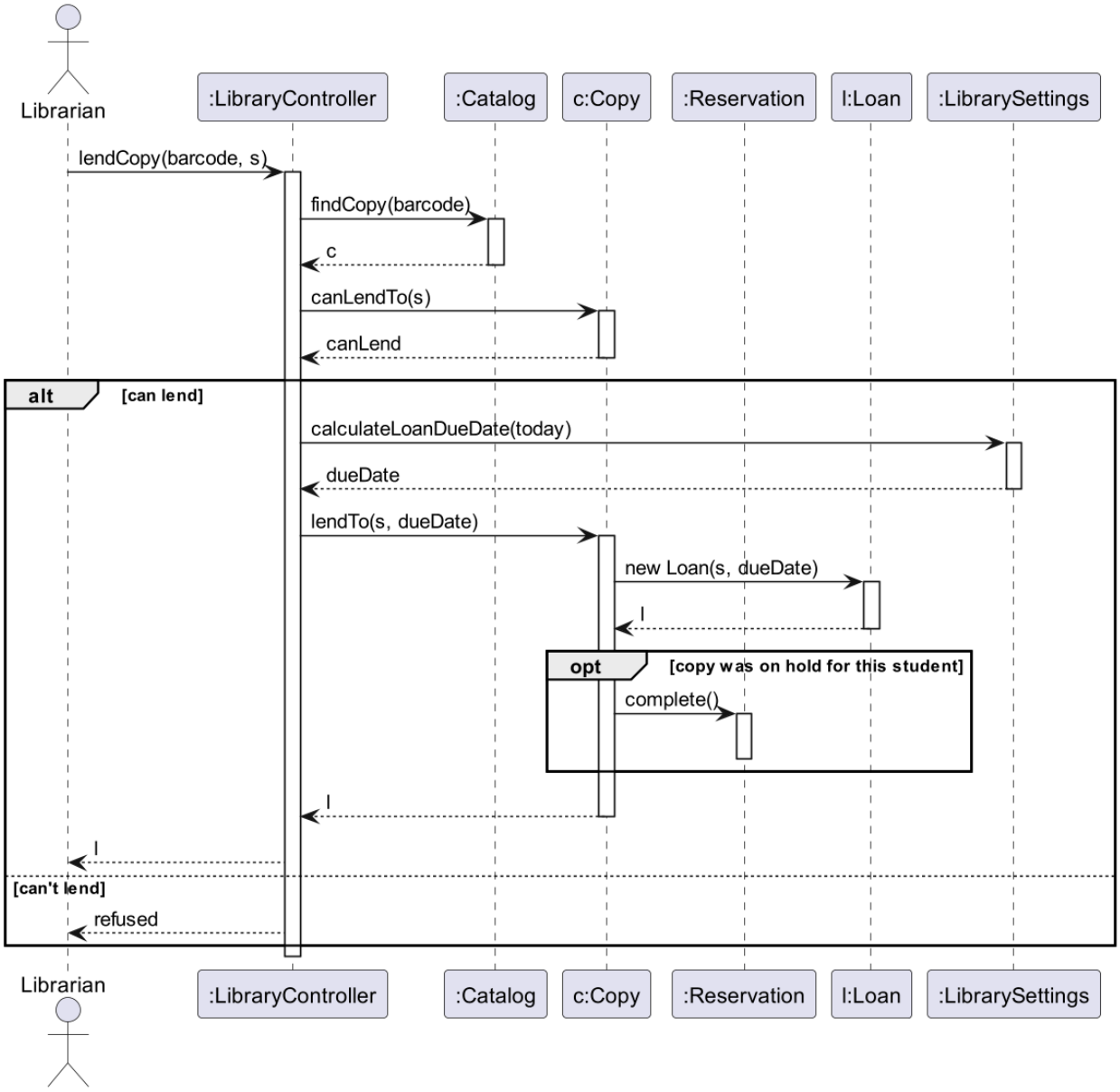
- Online payment of fines
- E-books / digital lending
- SMS / Push notification
- Recommendations
- Multiple library branches
- Detailed authentication / SSO
- Administrator role, staff management and usage reports
- Loan renewals / extensions (a due date cannot be extended)
- Recording which staff member handled a loan or waived a fine (staff are actors, not stored in the system)

3. Class Diagram

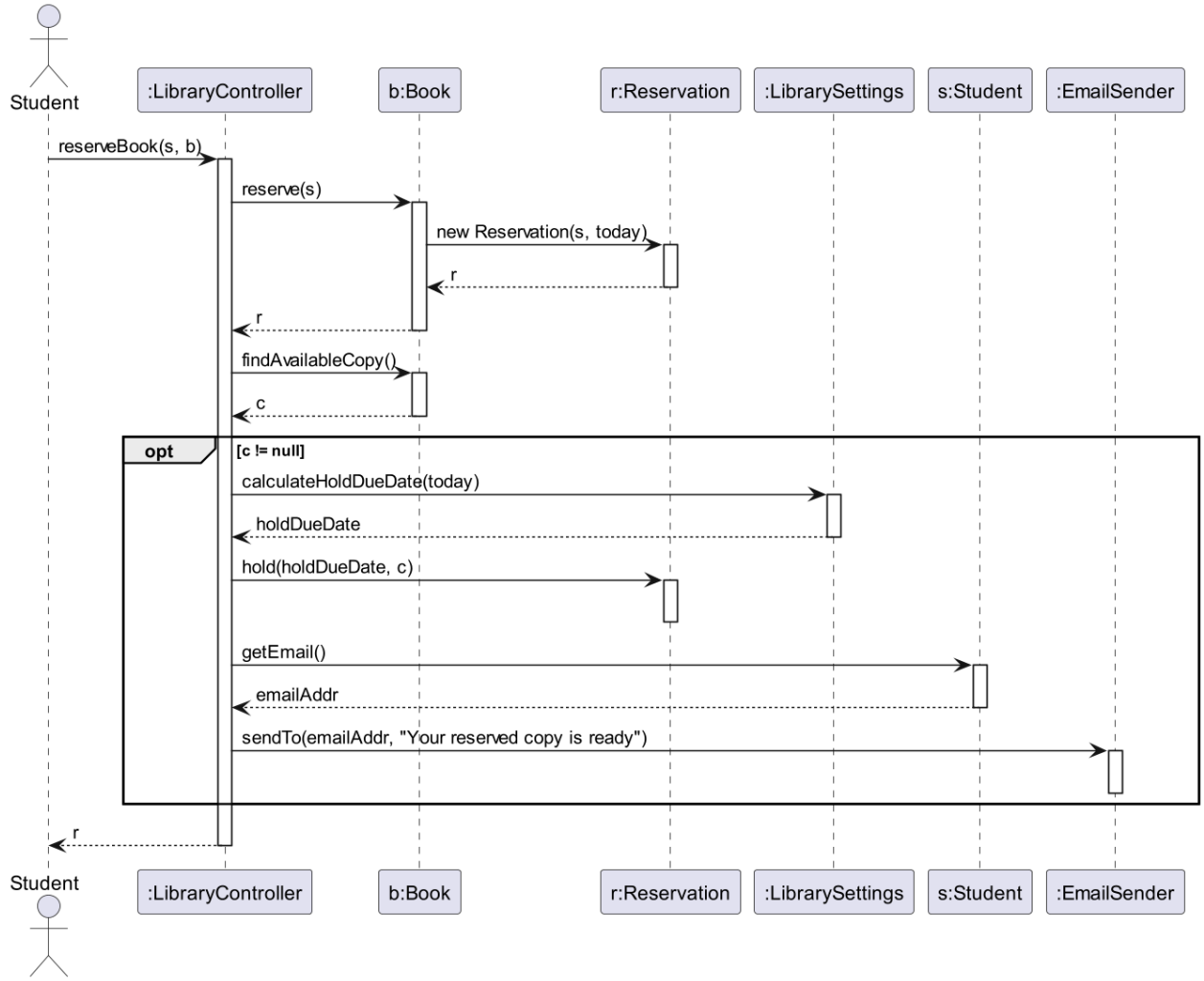


4. Sequence Diagrams

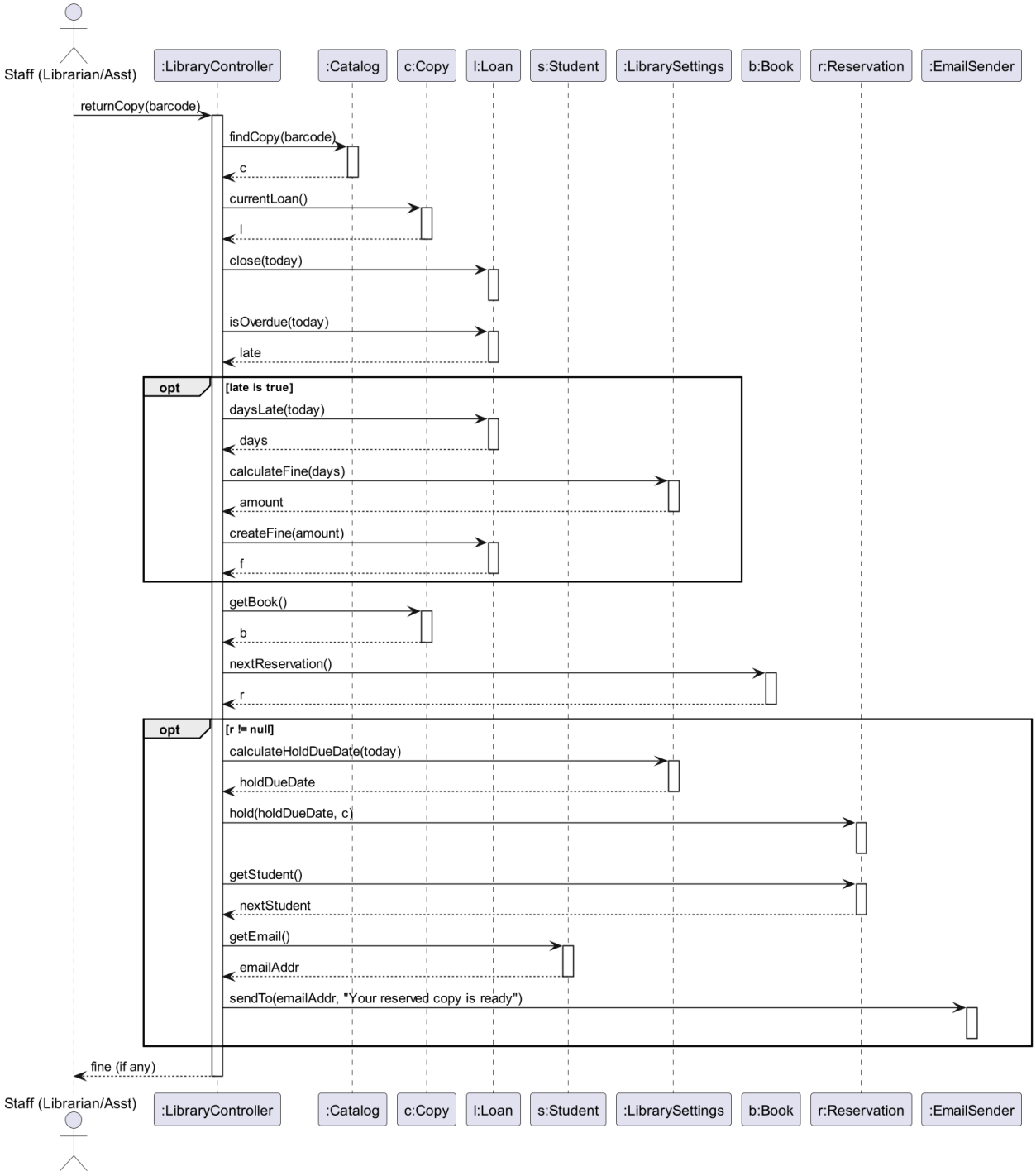
1. Librarian hands out a copy



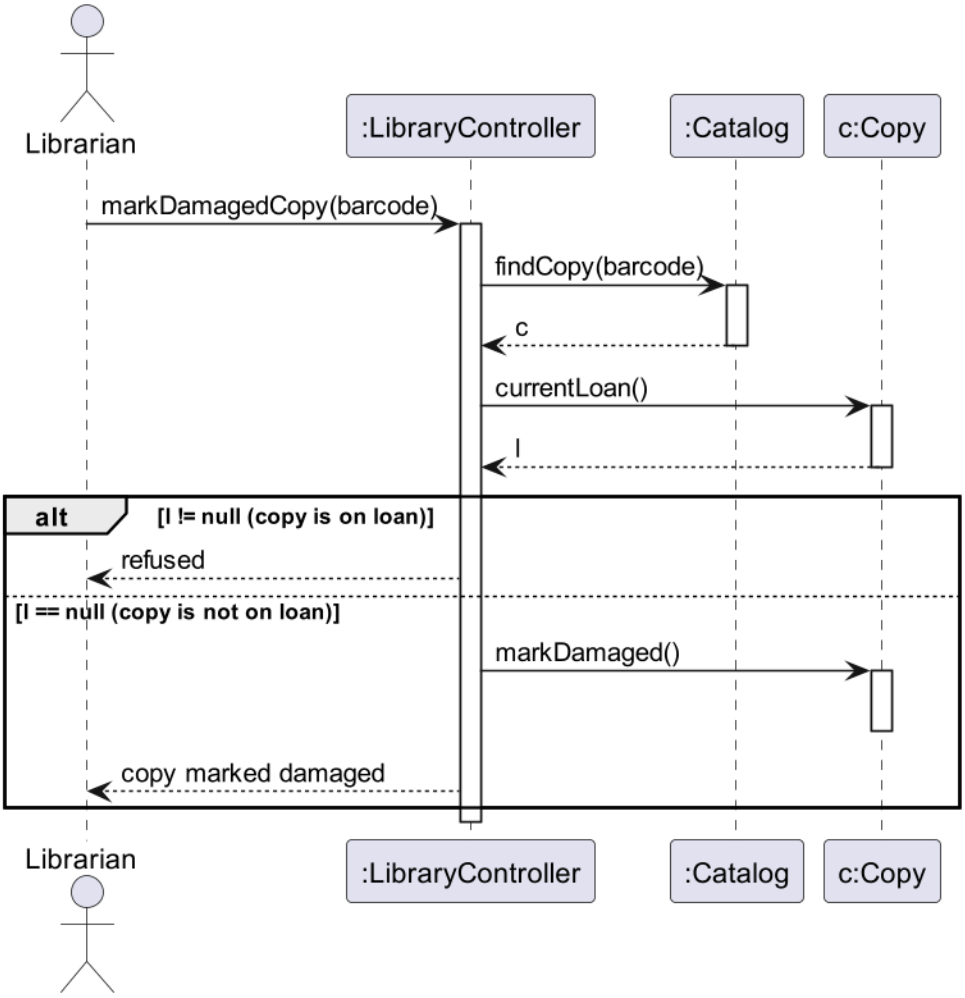
2. Student reserves a book



3. Student returns a book



4. Librarian marks a damaged copy



5. Design Risks and Planned Patterns

Design Risk 1: Email is hard-wired into the system

Right now `LibraryController` and `ReminderScheduler` both call `EmailSender` directly, as the two "1" -- "1" links in the class diagram show. That works as long as we only ever send email through one provider. But SMS and push notifications are out of scope for now, not forever, and the library could also switch to an email provider with a different API. Either change would mean editing both classes.

We plan to use the **Adapter** pattern. The controller and the scheduler would talk to a `NotificationSender` interface instead of `EmailSender`, and an `EmailAdapter` would wrap the actual provider. Adding SMS later would then just mean adding one more adapter, without touching the classes that send notifications. This follows the Dependency Inversion and Open/Closed principles, and protects the system from changes in how we notify students (Protected Variations).

Design Risk 2: Two copies of the library settings

`LibrarySettings` holds the loan period, hold period and fine rate, and both `LibraryController` and `ReminderScheduler` use it. Librarians can change these settings while the system is running. If the controller and the scheduler each ended up with their own `LibrarySettings` object, a change made at the desk wouldn't reach the scheduler. For example, a librarian changes the hold period from 3 to 5 days, but holds passed on by the scheduler overnight still get 3. The class diagram shows that both classes need the settings, but it can't show whether they share the same object.

We plan to use the **Singleton** pattern. We will create one instance of `LibrarySettings`, and every class that needs the settings uses that same instance. It is created once at start-up and passed to the controller and the scheduler (Dependency Injection). That way there is only ever one set of library rules, and a change made at the desk is what the scheduler sees.

Design Risk 3: One controller doing everything

`LibraryController` receives every request in the system, from the student's screen, the desk and the librarian's tools, so it has 19 operations covering searching, reservations, lending, returns, fines, books, copies and settings. It doesn't do the work itself (it passes everything on to the domain classes), but it mixes several unrelated groups of tasks in one class, and every new feature would make it bigger.

We plan to split it into three controllers, one per group of tasks (one per screen): a `StudentController` for searching and reservations, a `CirculationController` for lending, returns and fines, and a `CollectionController` for books, copies and settings. This follows the GRASP Controller principle (a controller per use case or session) and keeps each class focused (High Cohesion).